
Universidad de Guayaquil
Facultad de Ciencias Matemáticas y Físicas
Carrera de Software

Bitácora técnica

Prototipo web de BI asistido por IA

Estudiantes	Emily Dayan Robles Alvarado y Lesly Samay Velásquez Anchundia
Tema	Construcción asistida de un datamart, KPIs, insights y pronóstico
Repositorio local	/home/ceo/Proyectos/BI
Versión de la bitácora	0.1.5
Fecha de apertura	24 de agosto de 2026

Índice

1. Propósito y reglas de mantenimiento	1
2. Resumen del alcance vigente	1
3. Registro cronológico	1
3.1. 24 de agosto de 2026 - Inicio del repositorio y ambiente	1
3.1.1. Decisiones técnicas	2
3.1.2. Archivos y componentes principales	2
3.1.3. Incidentes y observaciones	3
3.2. 24 de agosto de 2026 - Formalización del Sprint 1 y manual técnico	3
3.3. 24 de agosto de 2026 - Gobierno Git, colaboradores y estabilidad DevOps	4
3.3.1. Incidentes y resolución	4
4. Matriz de verificación de la fase	5
5. Próximos pasos	5
6. Historial del documento	6

1. Propósito y reglas de mantenimiento

Este documento conserva la trazabilidad técnica del prototipo: decisiones, cambios, verificaciones, incidentes, evidencias y trabajo pendiente. Debe actualizarse al final de cada sesión significativa y antes de etiquetar una entrega.

Reglas:

- No eliminar incidentes ni resultados fallidos; agregar su resolución cuando exista.
- Registrar comandos y resultados relevantes sin copiar contraseñas, tokens o datos sensibles.
- Relacionar cada hito con un commit, pull request o etiqueta cuando GitHub esté disponible.
- Regenerar el PDF con `make logbook` y comprobar visualmente sus páginas.
- Mantener el archivo fuente `bitacora.tex` y el PDF generado bajo control de versiones.

2. Resumen del alcance vigente

El anteproyecto corregido define una prueba de concepto académica con una fuente activa SQL Server y AdventureWorks en modo de sólo lectura. El flujo futuro extraerá metadatos, solicitará propuestas estructuradas a un LLM, exigirá aprobación humana y validaciones determinísticas, ejecutará un ETL básico hacia PostgreSQL y mostrará cinco KPIs, tres visualizaciones, insights explicables y un pronóstico mensual mediante regresión lineal con MAPE y RMSE.

El anexo institucional exige módulos de seguridad, parámetros, negocio y reportes. Se acordó un RBAC mínimo con usuarios, roles, permisos y menús por rol. En la fase actual existe únicamente el límite arquitectónico; no hay autenticación ni lógica de negocio implementada.

3. Registro cronológico

3.1. 24 de agosto de 2026 - Inicio del repositorio y ambiente

Actividad	Estado	Resultado
Revisión del ante-proyecto	Completado	Se inspeccionaron las 11 páginas, incluyendo alcance, exclusiones, objetivos, metodología, cronograma y requisitos mínimos institucionales.
Arquitectura base	Completado	Se fijaron React con Vite y TypeScript, FastAPI, PostgreSQL interno/datamart y SQL Server AdventureWorks como fuente externa.
Docker-first	Completado	El host sólo requiere Docker Compose. Se definieron etapas de desarrollo, prueba y producción para frontend y backend.
Bases reproducibles	Completado	Se añadieron imágenes inicializadoras para PostgreSQL y SQL Server. AdventureWorks se restaura desde el respaldo oficial en un volumen vacío.

Actividad	Estado	Resultado
Seguridad	Completado	El paquete security y las reglas del RBAC quedaron documentados, sin modelos ni endpoints funcionales.
Observabilidad	Completado	Se añadieron endpoints /health/live y /health/ready , más una pantalla de estado en React.
DevOps	Completado	Se prepararon CI, CD, Dependabot, plantilla de pull request y publicación de imágenes en GHCR.
Guía de réplica	Completado	Se documentaron el entorno aislado desde cero y el consumo de imágenes publicadas.
GitHub remoto	Completado	Repositorio privado LFXAS/prototipo-bi-ia , rama main publicada y remoto origin configurado.
Verificación completa	Completado	El conjunto se reconstruyó desde volúmenes vacíos; los cuatro servicios quedaron saludables y las pruebas automatizadas finalizaron correctamente.

3.1.1. Decisiones técnicas

- El repositorio definitivo se ubica en **/home/ceo/Proyectos/BI**; no se codifican rutas absolutas en Compose, por lo que otras máquinas pueden clonarlo en cualquier ubicación.
- El Compose predeterminado crea red, bases y volúmenes aislados. Usa los puertos anfitrión 55432 y 51433 para no interferir con instancias existentes.
- Los volúmenes Docker no se exportan como imágenes. El estado inicial se reconstruye con respaldos públicos, scripts y futuras migraciones versionadas.
- **ApplicationIntent=ReadOnly** no es un control suficiente. El usuario **bi_reader** recibe lectura y denegaciones explícitas de escritura en SQL Server.
- CD publicará las imágenes propias en GitHub Container Registry con etiquetas de rama, versión y SHA.

3.1.2. Archivos y componentes principales

- **compose.yaml**: aplicación, PostgreSQL y SQL Server autocontenidos.
- **compose.release.yaml**: ejecución sin compilar, usando GHCR.
- **backend/**: API, configuración, sesión PostgreSQL, salud, Alembic y scaffolding modular.
- **frontend/**: interfaz React, proxy hacia la API, pruebas y Nginx de producción.
- **infra/**: inicialización de PostgreSQL, restauración de SQL Server y compilador LaTeX.
- **docs/replication-guide.md**: manual operativo paso a paso.
- **.github/workflows/**: integración y entrega continua.

3.1.3. Incidentes y observaciones

1. La carpeta generada inicialmente contenía un directorio `.git` vacío y no era todavía un repositorio válido. Se resolvió inicializando Git sobre `main`; el commit base es `ead11fb`.
2. El primer intento de generar `package-lock.json` falló porque npm trató de escribir en una caché de sólo lectura dentro del directorio personal. Se resolvió con la caché temporal `/tmp/bi-npm-cache`; el lockfile quedó generado sin vulnerabilidades reportadas.
3. Una verificación de Compose se lanzó desde la subcarpeta `frontend`; no encontró los archivos raíz. Se repitió desde la raíz y las configuraciones de desarrollo y publicación resultaron válidas.
4. La primera construcción del backend detectó formato pendiente y un import moderno requerido por Ruff. Se corrigieron ambos; Ruff, Mypy y dos pruebas Pytest finalizaron correctamente.
5. La primera prueba del frontend duplicó la inicialización de `jest-dom`. Se retiró la importación redundante; ESLint, Vitest y Vite finalizaron correctamente.
6. npm detectó vulnerabilidades en las versiones iniciales de Vite y Vitest. Se actualizaron a las versiones corregidas 7.3.6 y 3.2.7; el lockfile registra cero vulnerabilidades.
7. La primera salud del frontend consultó `localhost`, que BusyBox resolvió como IPv6 mientras Vite escuchaba en IPv4. Se cambió la sonda a `127.0.0.1` y el contenedor quedó saludable.
8. La primera restauración creó `bi_reader`, pero una denegación demasiado amplia de `CONTROL` también bloqueó el permiso subordinado de conexión. Se eliminó esa denegación, la sonda de salud pasó a probar el usuario real y se reconstruyó todo desde volúmenes vacíos.
9. La primera restauración de AdventureWorks requiere red y puede tardar varios minutos. Los reinicios posteriores reutilizan el volumen persistente.
10. La primera ejecución de Actions finalizó correctamente, pero advirtió que varias acciones todavía declaraban Node.js 20. Se actualizaron a las versiones oficiales vigentes: `checkout@v7`, `upload-artifact@v7` y acciones Docker de generaciones 4, 6 y 7.
11. La búsqueda autorizada de tres correos no devolvió cuentas públicas y la API de colaboradores exige nombres de usuario. Las invitaciones quedan pendientes hasta obtener los identificadores GitHub o iniciar sesión en la interfaz web.

3.2. 24 de agosto de 2026 - Formalización del Sprint 1 y manual técnico

- Se formalizó la fase de ambiente como **Sprint 1: ambiente reproducible, repositorio y DevOps**, con objetivo, backlog, criterios de aceptación, revisión, incidencias y retrospectiva.
- Se creó el informe académico `docs/sprints/sprint-01-entorno.tex` y su PDF, acompañado por capturas reales del frontend y OpenAPI.
- Se creó el manual `docs/manual-tecnico/manual-tecnico.tex` y su PDF con instalación, configuración, réplica, operación, pruebas, CI/CD, seguridad, diagnóstico y recuperación.
- `make docs` regenera los tres documentos con la imagen LaTeX del proyecto; `make verify` los incluye en la definición de terminado.
- CI compila, compara y publica los tres PDF como un único artefacto de documentación técnica.

3.3. 24 de agosto de 2026 - Gobierno Git, colaboradores y estabilidad DevOps

Actividad	Estado	Resultado
Colaboradores GitHub	Completado	Se verificaron LeslyVelasquez y EmyRoblesBerry; ambas cuentas tienen permiso de escritura en LFXAS/prototipo-bi-ia.
Ramas permanentes	Completado	<code>develop</code> quedó como rama predeterminada de integración y <code>main</code> como rama estable de publicación. El trabajo usa <code>feature/*</code> , <code>fix/*</code> , <code>docs/*</code> o <code>chore/*</code> .
Visibilidad	Completado	Por autorización expresa, el repositorio cambió de privado a público para habilitar las reglas de protección sin contratar todavía GitHub Pro. La visibilidad de los paquetes GHCR se gestiona por separado.
Protección de ramas	Completado	<code>develop</code> y <code>main</code> exigen pull request, una aprobación, conversaciones resueltas, rama actualizada y ocho controles de CI; force-push y eliminación están deshabilitados.
Flujo automático	Completado	CI valida ambas ramas y rechaza promociones a <code>main</code> cuyo origen no sea <code>develop</code> ; CD continúa limitado a <code>main</code> y etiquetas <code>v*</code> .
Dependabot	Completado	Las ramas <code>dependabot/*</code> se identificaron como temporales. npm agrupa sólo parches y cambios menores; los saltos mayores quedan fuera de las actualizaciones rutinarias.
Reinicio SQL Server	Completado	El inicializador espera que AdventureWorks esté en línea antes de configurar el lector y crear la marca de salud. Un reinicio con volumen existente terminó con cuatro servicios saludables y 71 tablas.
Manual y guía	Completado	README, guía de réplica, flujo Git y manual técnico describen comandos, promociones, protecciones, Dependabot y recuperación.
Verificación final	Completado	<code>make verify</code> validó Compose, política de ramas, backend, frontend y los tres PDF; Actionlint 1.7.12 validó los flujos, todo dentro de Docker.

3.3.1. Incidentes y resolución

1. El PR automático del frontend agrupó 18 actualizaciones y falló durante `npm ci`. Se reprodujo el error: TypeScript 7.0.2 era incompatible con `typescript-eslint` 8.67.0, que admite versiones menores que 6.1. Se corrigió la política para no agrupar ni proponer saltos mayores rutinarios; el PR incompatible debe cerrarse sin fusionar.
2. La captura de Actions también mostraba avisos por acciones basadas en Node.js 20. La rama `main` ya contenía las generaciones vigentes de `checkout`, `upload-artifact` y las acciones Docker; los avisos procedían de una rama Dependabot atrasada dos commits.

3. El primer intento de proteger el repositorio privado devolvió HTTP 403 porque el plan actual no incluye protección de ramas privadas. El usuario autorizó volver público el repositorio; la misma configuración se aplicó correctamente a **develop** y **main**.
4. Una prueba adicional **make lint** se detuvo porque SQL Server estaba no saludable después de un reinicio. El motor aceptaba conexiones, pero AdventureWorks seguía recuperándose cuando el inicializador intentó configurarla y dejó ausente la marca de disponibilidad. Se añadió una espera explícita a la base, se recreó el contenedor sin borrar el volumen y la prueba de reinicio pasó.

4. Matriz de verificación de la fase

Control	Estado	Evidencia esperada
Compose autocontenido válido	Completado	<code>docker compose config -quiet</code>
Compose de imágenes válido	Completado	<code>docker compose -f compose.release.yaml config -quiet</code>
Backend calidad y pruebas	Completado	Ruff, Mypy y 2 pruebas Pytest dentro de Docker
Frontend calidad y pruebas	Completado	ESLint, 1 prueba Vitest y build de Vite dentro de Docker
PostgreSQL disponible	Completado	Esquemas <code>app</code> y <code>mart</code> ; endpoint <code>/health/ready</code> en <code>ok</code>
AdventureWorks restaurada	Completado	71 tablas; <code>bi_reader</code> : lectura 1 e inserción 0
Interfaz disponible	Completado	Frontend saludable en el puerto 5173 y API accesible mediante proxy
PDF legible	Completado	Todas las páginas renderizadas e inspeccionadas sin cortes ni solapamientos
Git local	Completado	develop para integración, main para entregas y ramas enfocadas por pull request
GitHub/GHCR	Completado	Repositorio público, dos colaboradoras con escritura, ramas protegidas y cinco imágenes publicadas en GHCR

5. Próximos pasos

1. Revisar individualmente los pull requests de Dependabot que queden abiertos en **develop**; no fusionar ninguno sin CI aprobada.
2. Congelar una etiqueta de ambiente inicial y registrar los digest de las imágenes.
3. Iniciar la siguiente iteración con migraciones del RBAC y parámetros, sin adelantar módulos de negocio.

6. Historial del documento

Versión	Fecha	Cambio
0.1.0	24-08-2026	Apertura de la bitácora y registro de la fase de ambiente, Docker, Git y DevOps.
0.1.1	24-08-2026	Prueba integral desde volúmenes vacíos, corrección del usuario lector e inicialización del repositorio Git.
0.1.2	24-08-2026	Ejecución desde la ruta definitiva y validación de la sesión GitHub de LFXAS.
0.1.3	24-08-2026	Repositorio privado, primera CI/CD exitosa, imágenes GHCR y mantenimiento de Actions.
0.1.4	24-08-2026	Informe formal del Sprint 1, manual técnico, capturas e integración de todos los PDF en CI.
0.1.5	24-08-2026	Colaboradores, estrategia <code>develop/main</code> , protección, control de ramas, Dependabot y reinicio seguro de SQL Server.